

Inductive Types, Structures, Typeclasses, and Termination in Lean

Lean deep dive: data, proof, interfaces, and recursive definitions

DSA5210 – Session 3

Ma Jiajun - May 20, 2026

Structures Package Data and Proof Fields

A structure Packages Data

`structure`

A structure declares a new type with named fields, a constructor, projections, and printable values when `Repr` is derived.

- `Point2D.mk` constructs points.
- `<1, 1>` uses anonymous constructor syntax.
- `p.x` projects a field.

```
structure Point2D where
  x : Int
  y : Int
  deriving Repr

#check Point2D.mk
-- Point2D.mk (x y : Int) : Point2D

#eval Point2D.mk 1 2
-- { x := 1, y := 2 }

def p : Point2D where
  x := 3
  y := -2

def q : Point2D := {1, 1}

#print q
-- def q : Point2D := { x := 1, y := 1 }

#eval q
-- { x := 1, y := 1 }

#eval p.x + p.y
-- 1
```

Dot Notation Gives Method-Style Calls

```
def Point2D.sumxy (p : Point2D) : Int :=  
  p.x + p.y  
  
def Point2D.add (p q : Point2D) : Point2D :=  
  {p.x + q.x, p.y + q.y}  
  
#eval p.sumxy  
-- 1  
  
#eval p.add q  
-- { x := 4, y := -1 }
```

OOP analogy

`p.sumxy` looks like a method call.

Lean meaning

`Point2D.sumxy` is an ordinary function; dot notation inserts `p` as the first argument.

ClosedInterval.valid Is a Proof Field

A structure can mix computational fields and proof fields.

- `left` and `right` are data.
- `valid` is proof that $\text{left} \leq \text{right}$.
- The invariant is checked when the value is constructed.

```
import Mathlib

structure ClosedInterval where
  left : Nat
  right : Nat
  valid : left ≤ right

variable (n : Nat)

def I1 : ClosedInterval where
  left := 3
  right := 5
  valid := by decide

def intervalFrom (n : Nat) : ClosedInterval where
  left := n
  right := n + 2
  valid := by simp only [le_add_iff_nonneg_right, zero_le]

#check intervalFrom
-- intervalFrom (n : Nat) : ClosedInterval

#check (intervalFrom n).valid
```

Stored Proof Fields Can Be Used Later

```
example (I : ClosedInterval) :  
  ∃ i, I.left ≤ i ∧ i ≤ I.right := by  
  use I.left  
  constructor  
  · simp only [Std.le_refl]  
  · exact I.valid
```

Witness

Choose `i = I.left`.

Stored proof

The upper bound proof is exactly `I.valid`.

Fin n Is Value Plus Bound Proof

```
import Mathlib

#check Fin

#print Fin
-- structure Fin (n : Nat) : Type
-- fields: val : Nat, isLt : val < n

def secondOfFive : Fin 5 :=
  {2, by decide}

-- Expected error:
-- #check (2 : Fin 5) = (2 : Fin 6)

#check (2 : Fin 5) = (2 : Nat)
-- ↑2 = 2 : Prop

#eval (Finset.univ : Finset (Fin 5))
-- {0, 1, 2, 3, 4}

#check (2 : Fin 5)
-- 2 : Fin 5

#synth OfNat (Fin 5) 2
-- Fin.instOfNat
```

Interpretation

A value of ``Fin 5`` is a natural number plus a proof that it is less than ``5``.

Type boundary

``Fin 5`` and ``Fin 6`` are different types; comparison with ``Nat`` uses coercion.

Classes and Instance Search

Classes are structures Lean can find automatically.

The Problem After Overloaded Notation

Lean sees an overloaded symbol

``42 + "hi"` asks for an addition interface between ``Nat`` and ``String``.

Lean searches for an instance

If no matching instance exists, elaboration fails.

ordinary OOP class: object carries methods and state

Lean class: type has an interface, supplied separately by an instance

Notation Elaborates to Terms: Addition

```
#eval 42 + 2
-- 44

#check HAdd
-- HAdd (alpha : Type u) (beta : Type v)
--   (gamma : outParam (Type w)) : Type _

#check HAdd.hAdd
-- [HAdd alpha beta gamma] → alpha → beta → gamma

#synth HAdd Nat Nat Nat
-- instHAdd
```

`+`

Uses an `HAdd` instance.

Point

The same symbol can mean different operations on different types.

Custom HAdd Instance

```
/-  
instance haddnatstring1 : HAdd Nat String Nat where  
  hAdd := fun a b => a + b.length  
-/  
  
instance haddnatstring2 : HAdd Nat String String where  
  hAdd := fun a b => (toString a) ++ b  
  
-- #synth HAdd Nat String Nat  
#synth HAdd Nat String String  
-- haddnatstring2  
  
#eval 42 + "hi"  
-- "42hi"
```

The result type is part of the `HAdd` interface: `HAdd Nat String String` is different from `HAdd Nat String Nat`.

Coercion and Expected Type

```
instance stringToNat : Coe String Nat where  
  coe s := s.length
```

```
#eval ("hi" : Nat)  
-- 2
```

```
#eval (42 + "hi" : Nat)  
-- 44
```

```
#eval (42 + "hi" : String)  
-- "42hi"
```

Expected type

`String` asks for the custom `HAdd Nat String String` instance.

`Coe String Nat`

`Nat` lets Lean coerce `"hi"` to its length, then use ordinary `Nat + Nat`.

Numeric Literals Use OfNat

```
#check OfNat
-- OfNat ( $\alpha$  : Type u) : Nat  $\rightarrow$  Type u

#synth OfNat (Fin 5) 2
-- Fin.instOfNat

#check (2 : Fin 5)
-- 2 : Fin 5
```

``OfNat (Fin 5) 2``

The expected type ``Fin 5`` tells Lean how to interpret the numeral ``2``.

Not ``Coe Nat (Fin 5)``

``OfNat`` handles numeric literals. ``Coe A B`` handles automatic conversion from values of type ``A`` to values of type ``B``.

Notation Elaborates to Terms: Order

```
#check LE
-- LE (alpha : Type u) : Type u

#check LE.le
-- [LE alpha] → alpha → alpha → Prop

#synth HAdd Nat Nat Nat
-- instHAdd

#synth LE Nat
-- instLENat
```

``≤``

Uses an ``LE`` instance.

Instance search

``#synth LE Nat`` asks Lean which order interface is available for ``Nat``.

Mathlib's Preorder Extends LE

```
#check Preorder
-- Preorder (alpha : Type u) : Type u

#check le_refl
-- le_refl [Preorder alpha] : a ≤ a

#check le_trans
-- le_trans [Preorder alpha] :
--   a ≤ b → b ≤ c → a ≤ c
```

A preorder is a relation $\leq$ that is reflexive and transitive.

Terminology

Lean says `extends`; OOP usually says inheritance.

- ``LE alpha`` supplies the relation.
- ``LT alpha`` supplies strict order notation.
- ``le_refl`` supplies reflexivity.
- ``le_trans`` supplies transitivity.

Preorder Examples

Natural numbers

Ordinary numerical $\leq$.

Sets

Subset inclusion.

Tasks

$a \leq b$ means a can reach b in a dependency graph.

Conversions

$a \leq b$ means a can be converted into b .

If the graph has cycles, two different tasks may reach each other: preorder, not necessarily partial order.

Generic Theorems Ask for a Class

```
example [Preorder alpha] (a : alpha) : a ≤ a :=  
  le_refl
```

```
example [Preorder alpha] {a b c : alpha}  
  (hab : a ≤ b) (hbc : b ≤ c) : a ≤ c :=  
  le_trans hab hbc
```

- Parentheses `(x : A)` introduce ordinary explicit data.
- Square brackets `[C alpha]` introduce typeclass assumptions.
- Lean tries to fill square-bracket arguments by instance search.

Diamond Inheritance: Two Paths to One Interface

```
namespace DiamondDemo

class HasLabel (α : Type) where
  label : String

class AsNat (α : Type) extends HasLabel α where
  n : Nat

class AsBool (α : Type) extends HasLabel α where
  b : Bool

inductive Token where
  | token
```

HasLabel / \ AsNat AsBool \ /
Token

Question

If both parents provide a label, what does generic code get when it asks only for `HasLabel Token`?

Diamond Inheritance: Instance Search Chooses a Path

```
instance : AsNat Token where
  label := "Nat path"
  n := 0

instance : AsBool Token where
  label := "Bool path"
  b := true

#check (inferInstance : HasLabel Token)
-- inferInstance : HasLabel Token

#eval (inferInstance : HasLabel Token).label
-- "Bool path"
```

Problem

The two inheritance paths carry different `HasLabel` dictionaries.

Lesson

Design class hierarchies so shared inherited interfaces are coherent.

Inductive Types

Constructors are generation rules.

Constructors Generate Values

An inductive type is generated by its constructors.

- Constructors are the canonical ways to build values.
- Every value is built from finitely many constructor applications.
- To consume an inductive value, split into the constructor cases.

```
inductive MyBool where
  | false : MyBool
  | true  : MyBool

#check MyBool.false
-- MyBool.false : MyBool

#check MyBool.true
-- MyBool.true  : MyBool
```

Binary Trees Use nil and node

Generation rules

- ``nil : MyTree alpha``.
- If ``a : alpha`` and ``l r : MyTree alpha``, then ``node a l r : MyTree alpha``.
- There are no other basic ways to make a ``MyTree alpha``.

```
inductive MyTree (alpha : Type) where
| nil : MyTree alpha
| node : alpha → MyTree alpha → MyTree alpha → MyTree alpha
deriving Repr
```

Concrete Binary Trees

`t1`

```
  3  
 / \  
nil nil
```

`leftOnly`

```
  2  
 / \  
1  nil
```

`rightOnly`

```
  2  
 / \  
nil 3
```

```
namespace MyTree
```

```
def t1 : MyTree Nat :=  
  node 3 nil nil
```

Bad Tree Definition: leaf and node

```
inductive BadTree (alpha : Type) where
  | leaf : alpha → BadTree alpha
  | node : BadTree alpha → BadTree alpha → BadTree alpha
  deriving Repr
```

What it defines

A full binary tree: every internal node has both left and right subtrees.

What it cannot express

A node with only a left child, or only a right child.

The code typechecks, but the datatype does not match the intended ordinary binary tree.

Constructors Also Determine Proof Shape

Computation side

Pattern matching follows the constructor list.

One branch for ``nil``, one branch for ``node``.

Proof side

Induction follows the same constructor list.

Recursive constructor arguments produce induction hypotheses.

Induction is not only for natural numbers. Every inductive type has its own induction principle.

Natural-Number Induction

Nat Is Generated by zero and succ

```
-- Conceptual shape:  
-- inductive Nat where  
--   | zero : Nat  
--   | succ : Nat → Nat
```

```
0 : Nat succ 0 : Nat succ (succ 0)  
: Nat ...
```

Natural-number induction is the proof principle generated by these two constructors.

Natural-Number Induction Principle

To prove $P\ n$ for every $n : \text{Nat}$:

1. prove $P\ 0$,
2. prove $P\ (n + 1)$, assuming $P\ n$.

The induction hypothesis comes from the recursive argument of the successor constructor.

```
-- Conceptual type:  
P 0  
→ (forall n, P n → P (n + 1))  
→ forall n, P n
```

The Proposition Family Is the Motive

For the theorem below, the proposition family is:

$$P\ i := i + 1 = 1 + i$$

Induction proves $P\ i$ for an arbitrary $i : \text{Nat}$.

```
theorem add_one_eq_one_add (i : Nat) :  
  i + 1 = 1 + i := by  
  induction i with  
  -- two proof branches
```

induction i with Creates Two Branches

`zero` branch

```
goal: P 0
```

```
goal: 0 + 1 = 1 + 0
```

`succ` branch

```
i : Nat  
ih : P i  
goal: P (i + 1)
```

```
ih : i + 1 = 1 + i  
goal: (i + 1) + 1 = 1 + (i + 1)
```

Base Case: Computation Closes the Goal

```
theorem add_one_eq_one_add (i : Nat) :  
  i + 1 = 1 + i := by  
  induction i with  
  | zero =>  
    rfl
```

Goal

``0 + 1 = 1 + 0``

Why ``rfl`` works

Both sides reduce by computation to the same natural number.

Successor Case: Use the Induction Hypothesis

```
| succ i ih =>
  calc
    (i + 1) + 1 = (1 + i) + 1 := by rw [ih]
    _ = 1 + (i + 1) := by rw [Nat.add_assoc]
```

`ih``

``i + 1 = 1 + i``

Successor goal

``(i + 1) + 1 = 1 + (i + 1)``

The induction hypothesis proves the predecessor statement; the successor proof still needs rewriting and associativity.

What Can Go Wrong in Induction

No `ih` in base case

The `zero` branch has no predecessor.

`ih` is not the goal

It proves the predecessor statement, not the successor statement.

Wrong motive

A weak proposition family gives a weak induction hypothesis.

Later generalization

Strong and well-founded induction give larger predecessor sets.

Recursion from Constructors

Nat Recursion: Odd Part

```
namespace Sharkovskii

def oddPart : Nat → Nat
| 0 ⇒ 0
| n + 1 ⇒
  if (n + 1) % 2 = 0 then
    oddPart ((n + 1) / 2)
  else
    n + 1
termination_by k ⇒ k
decreasing_by
  exact Nat.div_lt_self (Nat.succ_pos n) (by decide)
```

Mathematical idea

For a positive period n , write $n = 2^e \cdot o$, where o is odd.

termination_by

The measure is the input k . The recursive call must make this measure smaller.

Nat Recursion: Two-Adic Exponent

```
namespace Sharkovskii

def twoAdic : Nat → Nat
| 0 ⇒ 0
| n + 1 ⇒
  if (n + 1) % 2 = 0 then
    twoAdic ((n + 1) / 2) + 1
  else
    0
termination_by k ⇒ k
decreasing_by
  exact Nat.div_lt_self (Nat.succ_pos n) (by decide)
```

What it returns

`twoAdic n` counts how many factors of `2` have been stripped.

`decreasing\_by`

`Nat.div\_lt\_self` proves that dividing a positive natural number by `2` gives a smaller input.

Evaluating the Odd Part

- `oddPart 12 = 3`, because $12 = 2^2 \cdot 3$.
- `twoAdic 12 = 2`, the exponent of 2.
- `oddPart 40 = 5`, because $40 = 2^3 \cdot 5$.

Parentheses matter: ``oddPart ((n + 1) / 2)``
recurses on the input after division.

```
#eval oddPart 0
-- 0
#eval oddPart 12
-- 3
#eval twoAdic 12
-- 2
#eval oddPart 40
-- 5
#eval twoAdic 40
-- 3

end Sharkovskii
```

AI Can Easily Produce Bad Definitions

```
namespace Sharkovskii

def stripTwosAux : Nat → Nat → Nat
| 0, n ⇒ n
| fuel + 1, n ⇒
  if n % 2 = 0 && n ≠ 0 then
    stripTwosAux fuel (n / 2)
  else
    n

def oddPart (n : Nat) : Nat :=
  stripTwosAux n n
```

Problem

The helper `fuel` forces structural termination but hides the mathematical operation.

Lesson

Generated Lean must be read for mathematical intent, not only for local typechecking.

Tree Recursion: Depth and Size

```
namespace MyTree

def depth : MyTree alpha → Nat
| nil ⇒ 0
| node _ l r ⇒ Nat.max (depth l) (depth r) + 1

def size : MyTree alpha → Nat
| nil ⇒ 0
| node _ l r ⇒ 1 + size l + size r
```

``depth``

Combines two recursive results with maximum.

``size``

Combines two recursive results with addition.

Tree Recursion: Mirror

```
namespace MyTree

def mirror : MyTree alpha → MyTree alpha
| nil ⇒ nil
| node a l r ⇒ node a (mirror r) (mirror l)

#eval depth t3
-- 3
```

``mirror``

Recurse on both children and swap them.

Constructor shape

The ``node`` branch stores a value, a left subtree, and a right subtree.

Tree Structural Induction

```
namespace MyTree

theorem size_mirror (t : MyTree alpha) :
  size (mirror t) = size t := by
  induction t with
  | nil =>
    rfl
  | node a l r ihl ihr =>
    simp [mirror, size, ihl, ihr]
    omega

end MyTree
```

Structural induction comes from the constructors. Later, well-founded induction is stronger: it can use any relation with no infinite descending chain.

Termination

Recursive definitions become part of Lean's logic.

Structural Recursion Is the Easy Case

Lean accepts recursive definitions when recursive calls are visibly made on structurally smaller data.

The recursive call is on `n`, the smaller part of `n + 1`.

```
def factorial : Nat → Nat
| 0 ⇒ 1
| n + 1 ⇒ (n + 1) * factorial n

#eval factorial 5
-- 120
```

Nonterminating Recursion Is Rejected

Expected failure

The successor branch calls the function again on exactly the same input.

Logical reason

If arbitrary nonterminating definitions were accepted, the logic would become unsound.

```
-- Expected failure:  
-- def bad : Nat → Nat  
--   | 0 ⇒ 0  
--   | n + 1 ⇒ bad (n + 1)
```

Well-Founded Induction

General recursion needs a decreasing relation.

Accessibility Means Every Descent Is Handled

```
#check Acc
-- Acc (r : alpha → alpha → Prop) : alpha → Prop

#check Acc.intro
-- constructor for accessibility

#check WellFounded
-- WellFounded (r : alpha → alpha → Prop) : Prop
```

Relation orientation

$r\ y\ x$ means that y is a predecessor of x .

Well-founded

Every element is accessible: there is no infinite descending chain.

Well-Founded Induction

To prove $P\ x$ for every x : fix x
assume $P\ y$ for every predecessor y
of x prove $P\ x$

$(\text{forall } x, (\text{forall } y, r\ y\ x \rightarrow P\ y) \rightarrow P\ x) \rightarrow \text{forall } x, P\ x$

```
example {alpha : Type} {r : alpha → alpha → Prop}
  (wf : WellFounded r)
  (P : alpha → Prop)
  (step : forall x, (forall y, r y x → P y) → P x)
  (x : alpha) : P x :=
  wf.induction x step
```

Mathlib Tree: Induction by Node Count

```
#check Tree
-- Tree ( $\alpha$  : Type u) : Type u

#check Tree.node
-- Tree.node :  $\alpha \rightarrow \text{Tree } \alpha \rightarrow \text{Tree } \alpha \rightarrow \text{Tree } \alpha$ 

#check Tree.numNodes
-- Tree.numNodes : Tree  $\alpha \rightarrow \text{Nat}$ 

example (P : Tree  $\alpha \rightarrow \text{Prop}$ )
  (step :  $\forall t : \text{Tree } \alpha,$ 
    ( $\forall s : \text{Tree } \alpha,$ 
      Tree.numNodes s < Tree.numNodes t  $\rightarrow P s \rightarrow P t$ ) :
     $\forall t : \text{Tree } \alpha, P t := \text{by}$ 
  intro t
  exact (InvImage.wf Tree.numNodes wellFounded_lt).induction t step
```

Measure

`Tree.numNodes : Tree  $\alpha \rightarrow \text{Nat}$` sends a tree to a natural number.

`InvImage.wf`

Euclid's Algorithm Decreases in the Second Argument

```
def euclid : Nat → Nat → Nat
  | a, 0 ⇒ a
  | a, b + 1 ⇒ euclid (b + 1) (a % (b + 1))
termination_by _ b ⇒ b
decreasing_by
  exact Nat.mod_lt a (Nat.succ_pos b)

#eval euclid 30 12
-- 6
```

``termination_by``

Tells Lean to watch the second argument.

``decreasing_by``

Proves ``a % (b + 1) < b + 1`` using ``Nat.mod_lt``.

Important

This proof is about termination, not correctness of gcd.

termination_by and decreasing_by Syntax

```
def euclid : Nat → Nat → Nat
| a, 0 ⇒ a
| a, b + 1 ⇒ euclid (b + 1) (a % (b + 1))
termination_by _ b ⇒ b
decreasing_by
  exact Nat.mod_lt a (Nat.succ_pos b)
```

``_ b``

Names the arguments for the termination measure; ignore the first one.

``=> b``

The measure is the second argument.

``decreasing_by``

A proof block for the generated decrease obligations.

What Goes After decreasing_by

Current measure

The recursive branch has second argument ``b + 1``.

Recursive measure

The recursive call has second argument ``a % (b + 1)``.

Proof obligation

After ``decreasing_by``, fill a proof of the inequality Lean generated.

measure before: $b + 1$

recursive call: `euclid (b + 1) (a % (b + 1))`

measure after: $a \% (b + 1)$

needed: $a \% (b + 1) < b + 1$

one proof: `exact Nat.mod_lt a (Nat.succ_pos b)`

Strong Induction Is the Natural-Number Case

```
#check Nat.strong_induction_on
-- (n : Nat) →
--   (forall n, (forall m < n, p m) → p n) → p n

example (P : Nat → Prop)
  (step : forall n, (forall m, m < n → P m) → P n) :
  forall n, P n := by
  intro n
  exact Nat.strong_induction_on n step
```

- Ordinary induction gives the immediately previous case.
- Strong induction gives all smaller natural numbers.
- Well-founded induction generalizes this to any no-infinite-descent relation.

Notation and Sharkovskii Periods

Overloaded notation is an interface question.

Sharkovskii, Li, and Yorke



Oleksandr M. Sharkovskii

Ukrainian mathematician; also transliterated Sharkovsky.

Known here for the 1964 theorem ordering possible periods of interval maps.

Photo: Wikimedia Commons, CC BY-SA 3.0.



Tien-Yien Li

Taiwanese mathematician at Michigan State University; cited as T.-Y. Li.

Coauthor with Yorke of the 1975 paper "Period Three Implies Chaos".

Photo: Wikimedia Commons, CC BY-SA 4.0.



James A. Yorke

Mathematician and physicist at the University of Maryland.

Coauthor of "Period Three Implies Chaos" and central figure in chaos theory.

Photo: Wikimedia Commons, CC BY-SA 3.0.

Dynamics: Iteration

```
namespace Dynamics
```

```
def iterate {X : Type} (f : X → X) : Nat → X → X  
| 0, x ⇒ x  
| n + 1, x ⇒ f (iterate f n x)
```

Discrete dynamical system

A state space X with a self-map $f : X \rightarrow X$.

Orbit

$\langle x, f x, f (f x), \dots \rangle$ is represented by $\text{iterate } f \ n \ x$.

Dynamics: Exact Period

```
namespace Dynamics

def ReturnsAfter {X : Type} (f : X → X)
  (n : Nat) (x : X) : Prop :=
  iterate f n x = x

def IsPeriodicPoint {X : Type} (f : X → X)
  (n : Nat) (x : X) : Prop :=
  0 < n ∧ ReturnsAfter f n x

def HasExactPeriod {X : Type} (f : X → X)
  (n : Nat) (x : X) : Prop :=
  IsPeriodicPoint f n x ∧
  ∀ m : Nat, 0 < m → m < n → ¬ ReturnsAfter f m x

def HasPointOfExactPeriod {X : Type} (f : X → X)
  (n : Nat) : Prop :=
  ∃ x : X, HasExactPeriod f n x

end Dynamics
```

Return after `n` steps

`ReturnsAfter f 0 x` is always true.

Exact period

Positive return after `n` steps and no earlier positive return.

Point of exact period exists

`HasPointOfExactPeriod f n` means some point has exact period `n`.

Sharkovskii's Order Is Not Numerical Order

3, 5, 7, 9, ...
 $2 \cdot 3, 2 \cdot 5, 2 \cdot 7, \dots$
 $2^{2 \cdot 3}, 2^{2 \cdot 5}, 2^{2 \cdot 7}, \dots$
...
 $\dots, 2^3, 2^2, 2, 1, 0$

Mathematical role

The order describes forcing between possible exact periods of interval maps.

Lean role

The period relation is a good object for modeling notation and order interfaces.

References in the notes: Sharkovskii (1964); Li and Yorke, "Period Three Implies Chaos" (1975); CPP 2023 Lean 3 formalization.

A Lean 3 Formalization Exists

CPP 2023 proof pearl

A CPP 2023 proof pearl formalised Sharkovskii's theorem in Lean 3.

Human representation choice

`Nat`, positive periods, custom structures, predicates, relations, classes, and notation are design decisions.

Lean 4 teaching role

Here we use the theorem to isolate the first modeling decision: how to represent periods and the period order.

Possible theorem topic

This is one possible theorem-formalization topic, not a common assignment for everyone.

"Formalising Sharkovsky's Theorem (Proof Pearl)", CPP 2023.

Sharkovskii's Theorem

Theorem

Let I be an interval in the real line, let $f : I \rightarrow I$ be continuous, and let m, n be positive integers.

If there exists a point x of exact period m for f , and m comes before n in the Sharkovskii order, then there exists a point of exact period n for f .

Mathematical content

The order records which periods force which other periods for continuous interval maps.

Lean content

We model the period order, not the theorem about real intervals and continuous maps.

Li-Yorke: Period Three Implies Chaos

Period-forcing part

For a continuous interval map, the existence of a point of exact period 3 implies that for every positive integer k , there exists a point of exact period k .

Why Sharkovskii explains this

The number 3 is the first element in Sharkovskii's order.

What we are not formalizing here

Li-Yorke also proves a stronger chaotic-behavior statement. This lecture uses only the period-forcing part.

A Decidable Sharkovskii Relation

```
namespace Sharkovskii

abbrev sharkLE (m n : Nat) : Prop :=
  n = 0 ∨
  (1 < oddPart m ∧ oddPart n = 1) ∨
  (1 < oddPart m ∧ 1 < oddPart n ∧
    (twoAdic m < twoAdic n ∨
     (twoAdic m = twoAdic n ∧ oddPart m ≤ oddPart n))) ∨
  (oddPart m = 1 ∧ oddPart n = 1 ∧ n ≤ m)
```

Proposition, not Bool

``sharkLE m n`` is a proposition built from decidable arithmetic statements.

Non-powers of two

Compare ``twoAdic`` first, then ``oddPart`` only when the exponents match.

``0``

``n = 0`` makes ``0`` largest; powers of two use ordinary ``n ≤ m``.

Computed Examples Through decide

```
infix:50 " ≤ₛ " ⇒ sharkLE

#check sharkLE
-- Sharkovskii.sharkLE (m n : Nat) : Prop

#check (3 ≤ₛ 5)
-- 3 ≤ₛ 5 : Prop

#eval decide (3 ≤ₛ 5)
-- true
#eval decide (5 ≤ₛ 6)
-- true
#eval decide (6 ≤ₛ 5)
-- false
#eval decide (8 ≤ₛ 4)
-- true
#eval decide (4 ≤ₛ 8)
-- false
#eval decide (1 ≤ₛ 0)
-- true
#eval decide (0 ≤ₛ 1)
-- false
```

Natural decidability

No custom `Decidable` instance is needed;
Lean can decide the arithmetic proposition.

``1 ≤ₛ 0``

This is true because `0` is the largest element
in the augmented order.

Formal Theorem Statement Using the Order

```
def SharkovskiiTheoremStatement
  {I : Type}
  (IsContinuousIntervalMap : (I → I) → Prop) : Prop :=
  ∀ f : I → I, IsContinuousIntervalMap f →
    ∀ m n : Nat, 0 < n → m ≤s n →
      Dynamics.HasPointOfExactPeriod f m →
        Dynamics.HasPointOfExactPeriod f n

#check SharkovskiiTheoremStatement
-- Sharkovskii.SharkovskiiTheoremStatement
--   {I : Type} →
--   ((I → I) → Prop) → Prop
end Sharkovskii
```

Formal shape

Exact period m plus $m \leq_s n$ forces exact period n .

Positive target period

$0 < m$ is already inside
 $\text{HasPointOfExactPeriod } f \ m$; keep $0 < n$ to
exclude the trivial zero-step return.

Inheritance Through Order Structures

Stronger interfaces extend weaker ones.

PartialOrder Adds Antisymmetry

```
#check PartialOrder
-- PartialOrder (alpha : Type u) : Type u

#check le_antisymm
-- le_antisymm [PartialOrder alpha] :
--   a ≤ b → b ≤ a → a = b
```

A partial order is a preorder satisfying antisymmetry.

If $a \leq b$ and $b \leq a$, then $a = b$.

Graph Reachability Separates the Concepts

Reachability graph

Reflexive-transitive closure gives a preorder.

DAG reachability

No cycles, so mutual reachability forces equality.

preorder: a reaches b b reaches c therefore a reaches c

partial order: preorder plus no two distinct elements reach each other both ways

Lean's Built-In Order Hierarchy

```
#check Preorder
-- Preorder (alpha : Type u) : Type u

#check PartialOrder
-- PartialOrder (alpha : Type u) : Type u

#check Nat.instPreorder
-- Nat.instPreorder : Preorder Nat

#check Nat.instPartialOrder
-- Nat.instPartialOrder : PartialOrder Nat

#check LinearOrder
-- LinearOrder (alpha : Type u) : Type u
```

`Preorder` fields

`Preorder` already includes ` $\text{LE}$ `, ` $\text{LT}$ `, reflexivity, transitivity, and compatibility between ` $<$ ` and ` $\leq$ `.

Ask for the Weakest Interface

```
example [PartialOrder alpha] {a b : alpha}
  (hab : a ≤ b) (hba : b ≤ a) : a = b :=
  le_antisymm hab hba
```

Only reflexivity or transitivity?

Ask for `[Preorder alpha]`.

Need antisymmetry?

Ask for `[PartialOrder alpha]`.